☕ Java Mastery Cheat Sheet (Extended Edition)

1. Basics, Memory, & Control Flow

Execution: `.java` (Source) ➔ `javac` (Compiler) ➔ `.class` (Bytecode) ➔ `JVM` (Execution)
Memory: * Stack: Stores method frames, local primitive values, and object references. Cleared when methods return.

- Heap: Stores actual Objects and Arrays. Handled by the Garbage Collector (GC).

Your First Java Program (Hello World)

Every Java application needs an entry point. The JVM looks for the `main` method to start running your code.

```
public class HelloWorld {
    // The main method is the exact starting point of your program
    public static void main(String[] args) {
        System.out.println("Hello, World!"); // Prints text to the console
    }
}
```

Breaking it down:

- `public` : Accessible from anywhere (the JVM needs to be able to see it).
- `class` : Everything in Java must live inside a class. The filename must match the public class name (`HelloWorld.java`).
- `static` : Belongs to the class itself. The JVM can run this method without needing to create an object of `HelloWorld` first.
- `void` : This method does not return any value when it finishes.
- `main` : The specific method name the JVM searches for.
- `String[] args` : Allows you to pass command-line arguments when starting the app.

Core Data Types & Wrappers

Every primitive has a "Wrapper Class" used in Collections (Collections cannot hold primitives).

Type	Wrapper Class	Size	Example	Notes
int	`Integer`	32-bit	`int x = 5;`	Default integer type.

long	`Long`	64-bit	`long x = 99L;`	Append `L`. Use for DB IDs.
double	`Double`	64-bit	`double d = 3.14;`	Default decimal type.
boolean	`Boolean`	-	`boolean b = true;`	`true` or `false`.
char	`Character`	16-bit	`char c = 'A';`	Single quotes.
String	N/A (Object)	Obj	`String s = "Hi";`	Double quotes. Immutable.

Note: Autoboxing automatically converts primitives to wrappers (e.g., `Integer num = 5;`).

String Methods (Object, not primitive)

```
String s = "Hello World";
s.length();                    // 11
s.substring(0, 5);             // "Hello" (End index is exclusive)
s.equals("Hello World");       // true (Use this for content check)
s.equalsIgnoreCase("hello world");  // true
s.contains("World");           // true
s.replace("World", "Java");    // "Hello Java"
s.trim();                      // Removes leading/trailing spaces
s.split(" ");                  // Returns String array: ["Hello", "World"]
```

Control Flow & Loops

```
// Enhanced Switch (Java 14+) - No break needed, returns values
String type = switch (day) {
    case "SAT", "SUN" -> "Weekend";
    default -> "Weekday";
};

// Loops
for (int i = 0; i < 5; i++) { ... }          // Standard For
for (String item : list) { ... }             // Enhanced For-Each
while (condition) { ... }                    // While
do { ... } while (condition);                // Do-While (Runs ≥1 time)

// Loop Control
break;      // Exits the loop entirely
continue;   // Skips the rest of the current iteration and goes to the next
```

2. Object-Oriented Programming (OOP)

Rule: Everything lives in a `class`. The filename must match the `public class` name exactly.

Access Modifiers

Modifier	Class	Package	Subclass	World	Best Practice / Usage
private	✅	❌	❌	❌	Instance variables, internal helper methods.
(default)	✅	✅	❌	❌	Package-private. Rarely used deliberately.
protected	✅	✅	✅	❌	Base class variables meant for child classes.
public	✅	✅	✅	✅	Constructors, API methods, entry points.

Crucial OOP Keywords

- `this`: Refers to the current instance of the class (resolves shadowing: `this.name = name;`).
- `super`: Refers to the parent class. Used to call parent constructor (`super()`) or overridden methods (`super.doSomething()`). Must be the first line in a child constructor.
- `static`: Belongs to the class, not the instance. Called without creating an object (`Math.max()`).
- `final`: Cannot be changed.
  - `final int x`: Constant value.
  - `final class`: Cannot be extended.
  - `final method`: Cannot be overridden.

Inheritance (`extends`)

- Child inherits non-private members. Supports single inheritance only (one parent).
- Used for an "IS-A" relationship (Dog IS-A Animal).

Polymorphism

- Overriding (`@Override`): Runtime. Child provides its own implementation of a parent's method.
- Overloading: Compile-time. Multiple methods with the same name, but *different parameters* in the same class.

Interfaces vs Abstract Classes

Feature	Interface (`implements`)	Abstract Class (`extends`)
Concept	A Contract (CAN-DO)	A Base Type (IS-A)
Methods	Usually abstract (implicitly `public abstract`). Can have `default` methods.	Can have both abstract & concrete (implemented) methods.
State	Only `static final` constants. No instance variables.	Can have regular instance variables & constructors.
Limits	Class can implement multiple interfaces.	Class can extend only one parent class.

3. Advanced Core Concepts (Prep for Spring)

Enums (Enumerations)

Used for fixed sets of constants (e.g., Roles, Statuses). Better than using plain Strings.

```
public enum Role {
    USER, ADMIN, MODERATOR
}
// Usage:
Role myRole = Role.ADMIN;
if (myRole == Role.ADMIN) { ... }
```

`java.time` API (Modern Dates)

Crucial for database timestamps and JSON serialization in Spring.

```
LocalDate date = LocalDate.now();                    // 2024-03-15 (No time)
LocalDateTime dateTime = LocalDateTime.now();        // 2024-03-15T14:30:00
LocalDate parsed = LocalDate.parse("2024-12-25");    // String to Date
```

4. Collections Framework & Generics

Collections are dynamic, unlike standard arrays (`int[] = new int[5]`), which have a fixed size.

The Big Three (List, Set, Map)

Interface	Implementations	Ordering	Duplicates	Access By	Best For
List	`ArrayList`, `LinkedList`	✅ Ordered	✅ Allowed	Index `get(0)`	Dynamic lists, frequent reading.
Map	`HashMap`, `TreeMap`	❌ None	Keys ❌ / Vals ✅	Key `get("id")`	Fast Lookups, Caching, JSON-like data.
Set	`HashSet`, `TreeSet`	❌ None	❌ Never	`contains(val)`	Fast uniqueness checks, filtering duplicates.

Note: `TreeMap` and `TreeSet` automatically sort their elements, but are slower than Hash versions.

Quick Syntax & Generics `<T>`

Generics provide *compile-time* type safety (prevents `ClassCastException`).

```
// ArrayList (O(1) access)
List<String> list = new ArrayList<>();
list.add("Ali"); list.add(0, "Zara"); // Insert at index
list.get(0);  list.remove("Ali");

// HashMap (O(1) lookup)
Map<String, Integer> map = new HashMap<>();
map.put("Ali", 95);
map.get("Ali");
map.getOrDefault("Bob", 0);          // Returns 0 if "Bob" is missing
map.putIfAbsent("Ali", 100);         // Won't overwrite the 95
map.keySet();                        // Returns a Set of all keys
map.values();                        // Returns a Collection of all values

// HashSet (O(1) uniqueness)
Set<String> set = new HashSet<>();
set.add("Ali");
boolean added = set.add("Ali");      // Returns false, size remains 1
```

5. Exceptions & Error Handling

Prevent app crashes by gracefully handling unexpected runtime errors.

Checked vs Unchecked

- Unchecked (`RuntimeException`): Logic errors (`NullPointerException` , `IndexOutOfBoundsException`). Handling is optional. Fix your code instead.
- Checked (`Exception`): External issues out of your control (`IOException` , `SQLException`). Java forces you to handle via `try-catch` or declare via `throws` .

Try / Catch / Finally Structure

```
try {
    int result = 10 / Integer.parseInt("0");  // ArithmeticException
} catch (NumberFormatException e) {
    System.out.println("Invalid number format");
} catch (Exception e) {
    System.out.println("Catch-all fallback. Message: " + e.getMessage());
} finally {
    System.out.println("Always runs. Great for cleaning up resources.");
}
```

Try-With-Resources (Modern Standard)

Automatically closes resources (like files, scanners, DB connections) without needing a `finally` block.

```
try (Scanner sc = new Scanner(new File("data.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
} catch (FileNotFoundException e) {
    e.printStackTrace();
} // Scanner automatically closed here
```

Throwing Custom Exceptions (Spring Boot Standard)

```
// Definition
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String id) { super("User missing: " + id); }
}

// Usage in a Service class
public User getUser(String id) {
    if (userDoesNotExist) {
        throw new UserNotFoundException(id);  // Stops execution immediately
    }
    return user;
```

```
}
```

6. Modern Java (Java 8 - 21+)

Lambdas (Anonymous Functions)

Shorthand for interfaces with a single abstract method (Functional Interfaces). *Syntax:* `(params) -> { body }`

```
list. forEach(name  ->  System. out. println(name));
list. forEach(System. out: : println);  //  Method  reference  shortcut

//  Sorting  with  Lambda
Collections. sort(names,  (a,  b)  ->  a. compareTo(b));
```

Stream API (Declarative Data Processing)

Data pipelines. No loops required. Original list is never modified.

- Intermediate (Chainable, Lazy): `. filter()`, `. map()`, `. sorted()`, `. distinct()`, `. limit()`
- Terminal (Produces result, triggers stream): `. toList()`, `. count()`, `. findFirst()`, `. anyMatch()`

```
// Example: Get names of top CS students in uppercase
List<String>  topCsStudents  =  students. stream()
    . filter(s  ->  s. getMajor(). equals("CS"))      // Filter
    . filter(s  ->  s. getGpa()  >  3. 5)             // Filter
    . map(s  ->  s. getName(). toUpperCase())         // Transform object to String
    . sorted()                                        //  Alphabetical  order
    . toList();                                       //  Collect  (Java  16+)

//  Advanced  Collectors
Map<String,  List<Student>>  byMajor  =  students. stream()
    . collect(Collectors. groupingBy(Student: : getMajor));
```

Optional `<T>` (Null Safety)

A container object that may or may not contain a non-null value. Forces the developer to handle the `null` case explicitly.

```
Optional<Student>  studentOpt  =  repository. findById("S123");
```

```
// Check and get safely
if (studentOpt.isPresent()) {
    Student s = studentOpt.get();
}

// Functional alternatives (Preferred)
Student safe = studentOpt.orElse(new Student("Default"));
Student valid = studentOpt.orElseThrow(() -> new UserNotFoundException("S123"));
studentOpt.ifPresent(s -> System.out.println(s.getName()));
```

Records (Java 16+)

Immutable data carriers. Replaces boilerplate POJOs (Plain Old Java Objects). Perfect for Spring Boot DTOs (Data Transfer Objects). Auto-generates constructor, getters (without `get` prefix), `equals`, `hashCode`, and `toString`.

```
// One line replaces 30+ lines of code
public record StudentDTO(String name, int age, double gpa) {}

// Usage:
StudentDTO dto = new StudentDTO("Ali", 20, 3.9);
System.out.println(dto.name());  // Access property
```

7. The Ecosystem (Maven & JSON)

Maven (Build & Dependency Management)

Managed via the `pom.xml` file. Handles pulling down external libraries automatically.

- `pom.xml`: Contains `groupId`, `artifactId`, `version`, and `<dependencies>`.
- `mvn clean` - Deletes old compiled builds (`/target` folder).
- `mvn install` - Compiles, runs tests, and packages into a `.jar`.
- `mvn spring-boot:run` - Starts local embedded web server.

JSON & Jackson

JSON is the universal language of Web APIs. Spring Boot uses the Jackson library to auto-convert Java Objects to JSON (Serialization) and JSON to Java Objects (Deserialization).

JSON Format Rules: Keys must be double-quoted strings. No trailing commas.

```
{
  "name": "Ali",
```

```
  "gpa": 3.9,
  "active": true,
  "skills": ["Java", "Spring Boot"]
}
```

Jackson Under the Hood:

```
ObjectMapper mapper = new ObjectMapper();

// Java Object -> JSON String (Serialization)
String jsonStr = mapper.writeValueAsString(studentObj);

// JSON String -> Java Object (Deserialization)
Student obj = mapper.readValue(jsonStr, Student.class);
```

8. Spring Boot Core Annotations Preview

Since you're preparing for Spring Boot, recognize these patterns:

- `@SpringBootApplication`: Starts the app.